

Learning Targets

Knowledge: Explain how images are stored as number grids · Understand what an "edge" means in math · Trace the Laplacian filter formula step-by-step · Connect to real AI (face unlock, self-driving cars, medical imaging)

Skills: Write and read nested loops · Apply the formula $\text{center} \times 4 - (\text{up} + \text{down} + \text{left} + \text{right})$ · Predict code output before running · Identify boundary conditions

Connection: The same edge-detection idea powers face unlock on your phone, medical scanners, and self-driving car cameras.

Vocabulary – 5 Key Words

Word	Definition
Pixel	The smallest unit in a digital image – stores one brightness number (0 = black, 255 = white)
Grayscale Grid	A 2D list of numbers representing an image with brightness only – no color
Edge	A location where brightness changes <i>rapidly</i> between neighboring pixels – outlines shapes and boundaries
Filter / Kernel	A small math operation slid over every pixel, combining it with its neighbors (blur, sharpen, or edge-detect)
Nested Loop	A loop inside another loop – outer loop moves through <i>rows</i> (y), inner loop through <i>columns</i> (x)

Quick Check: `image[2][2] = 9` and all its neighbors are 5. Is pixel on an edge? How do you know?

Warm-Up (*Dot in your notebook – revisit at the end*)

1. How can a computer "see" a picture if it can only store *numbers*?
2. What might **change in the numbers** where a dark sky meets a bright building?
3. If you **blur** a picture, what happens to fine details in the number grid?

Images as Numbers

A digital image is just a **grid of numbers**. Each number is one pixel's brightness.

text

Our 5×5 mini-image (0 = dark, 9 = brightest):

```
[ 0, 0, 0, 0, 0 ]
[ 0, 5, 5, 5, 0 ]
[ 0, 5, 9, 5, 0 ] ← brightest pixel at center
[ 0, 5, 5, 5, 0 ]
[ 0, 0, 0, 0, 0 ]
```

What is an edge? A place where brightness jumps fast:

0 → 0 → 5 → 5 → 0 – the jump from 0 to 5 **is** an edge; the drop from 5 back to 0 is another.

Edges define **shapes** → shapes define **objects** → if AI finds edges, it can recognize letters, faces, stop signs, and tumors – even without knowing any colors.

The Filter Formula (Laplacian-style)

$\text{value} = \text{center} \times 4 - (\text{up} + \text{down} + \text{left} + \text{right})$

- **Result near 0** → center and neighbors are similar → flat, boring region
- **Result large** → center is very different from neighbors → **edge!**

Invented by Pierre-Simon Laplace in the 1700s – he had no idea it would one day unlock your phone.

Trace pixel by hand:

Variable	Value
<code>center = image[1][1]</code>	5
<code>up = image[0][1]</code>	0
<code>down = image[2][1]</code>	5
<code>left = image[1][0]</code>	0
<code>right = image[1][2]</code>	5
value = $5 \times 4 - (0 + 5 + 0 + 5)$	= 10 → edge!

The Code – Type Along

python

```
image = [
    [0, 0, 0, 0, 0],
    [0, 5, 5, 5, 0],
    [0, 5, 9, 5, 0],    # brightest!
    [0, 5, 5, 5, 0],
    [0, 0, 0, 0, 0],
]

filtered = []
for y in range(1, 4):    # rows 1, 2, 3
    new_row = []
    for x in range(1, 4): # cols 1, 2, 3
        center = image[y][x]
        up      = image[y-1][x]
```

```

    down = image[y+1][x]
    left = image[y][x-1]
    right = image[y][x+1]
    value = center*4 - (up + down + left + right)
    new_row.append(value)
    filtered.append(new_row)

```

print(filtered)

Read the code:

- range(1, 4) skips rows/cols 0 and 4 – they have no neighbor on one side (**boundary condition**)
- image[y-1][x] = the pixel *above* the current one
- If center = all neighbors → value = 0 (flat zone)
- If center = 9, neighbors = 5 → value = 9*4 - 20 = **16** (strong edge!)

Code questions:

1. Why does range(1, 4) skip row 0 and row 4?
2. What does image[y+1][x] refer to – above, below, left, or right?
3. If center = 5 and all 4 neighbors = 5, what is value? Does that make sense?
4. Calculate by hand: center = 9, all 4 neighbors = 5. What does a big positive answer mean?

Exercise A – Spot the Edge (*Fill the blanks*)

```

python
up    = image[____][x]      # Blank 1
down  = image[____][x]      # Blank 2
left  = image[y][____]      # Blank 3

```

Answers: Blank 1 = y-1 · Blank 2 = y+1 · Blank 3 = x-1

Bonus: Which cells in filtered will have the *highest* values? Draw the 3×3 grid and mark them.

Exercise B – Build Your Own Filter (*Predict first!*)

Use this new image – a **bright horizontal stripe** in the middle:

```

python
image = [
    [0, 0, 0, 0, 0],
    [0, 0, 0, 0, 0],
    [0, 8, 8, 8, 0],    # bright stripe
    [0, 0, 0, 0, 0],
    [0, 0, 0, 0, 0],
]

```

1. **Draw** the 3×3 filtered grid on paper and fill in your predicted values *before* writing code
2. Apply value = center*4 - (up+down+left+right) and build the filtered list
3. Which rows have the biggest values? Why?

Super challenge: After printing, add a loop that prints "EDGE!" next to any value greater than 15.

Kernel Types – Same Loop, Different Math

Kernel	What it does	Example use
Edge Detect (ours)	Highlights pixels very different from neighbors	Face unlock, object recognition
Blur (Average)	Replaces each pixel with the average of 9 neighbors	Noise removal, smoothing
Sharpen	Exaggerates differences from neighbors	Photo editors (Photoshop)

Convolution = sliding a kernel over every pixel with a nested loop. CNNs (Convolutional Neural Networks) *learn* thousands of kernels automatically by training on millions of photos. Your code is **layer 0 of deep learning**.

Real-World AI Connections

- **Face Unlock** – Phone finds edges (jawline, eye sockets, nose bridge) and matches them to a stored pattern
- **Self-Driving Cars** – Tesla/Waymo find road markings and pedestrians at **30+ frames/second** using edge filters
- **Medical Imaging** – Edge filters highlight tumor boundaries in MRI and X-ray scans, improving early cancer detection

Homework

Task 1 – Code: Design your own 5×5 grayscale image with at least one clear edge. Hand-draw the grid, write your predicted filtered values next to it, then run the code to check.

Task 2 – Reflect: Write **4–5 sentences** answering: "Why are edges so important for recognizing handwriting, faces, or road signs?"

- Explain what an edge is in your own words
- Give one specific example (handwriting, face, sign)
- Connect to the formula: how does the math "find" what you described?
- Look back at your warm-up answers – did today's code change your thinking?

Email a photo of your hand-drawn grid + code screenshot + reflection to hue.weng.88@gmail.com by next class.

Today's Lesson

<https://hue881.github.io/3/index.html>

<https://hue881.github.io/3/activity/index.html>

<https://github.com/hue881/AI-Coding-Day16/tree/main/3>